SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Computing Vision

Lab Task # 13

Submitted To: **Mr. Muhammad Azhar**

Student Name: **Habiba Bibi**

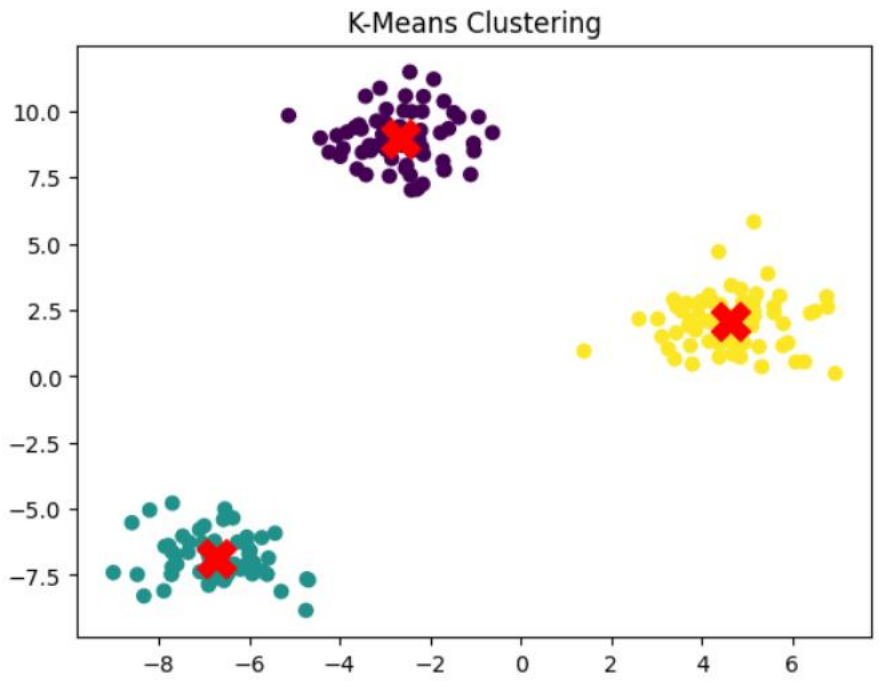
Reg. Number: **23108111**

Code Example 1: Customer Segmentation Using K-Means Clustering

ShopEase is a growing e-commerce platform that wants to personalize its marketing campaigns. The business team suspects that customers fall into natural segments based on spending behavior and visit frequency. However, these segments are not explicitly labeled in the data.

To uncover these hidden groups, the data science team applies K-Means clustering to identify customer segments based on purchasing patterns.

```
: # Code Example 1: Customer Segmentation Using K-Means Clustering
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, y = make_blobs(n_samples=200, centers=3, random_state=42)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

kmeans.fit(X)

y_kmeans = kmeans.predict(X)

plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis')

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=300,
    c='red',
    marker='X'
)

plt.title("K-Means Clustering")

plt.show()
```



Code Example 2: Dimensionality Reduction for Customer Segmentation Visualization using Principal Component Analysis (PCA)

```
# Code Example 2: Dimensionality Reduction for Customer Segmentation Visualization using Principal Component Analysis (PCA)
from sklearn.decomposition import PCA
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

pca = PCA(n_components=2)

X_reduced = pca.fit_transform(X)

df = pd.DataFrame(X_reduced, columns=['PC1', 'PC2'])

df['Cluster'] = y_kmeans

sns.scatterplot(
    data=df,
    x='PC1',
    y='PC2',
    hue='Cluster',
    palette='Set2'
)

plt.title("PCA Projection of Clustered Data")

plt.show()
```

Code Example 3: Customer Behavior Classification Using Neural Networks

```python
# Code Example 3: Customer Behavior Classification Using Neural Networks
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

clf = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    max_iter=1000,
    random_state=42
)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print(classification_report(y_test, y_pred))
```

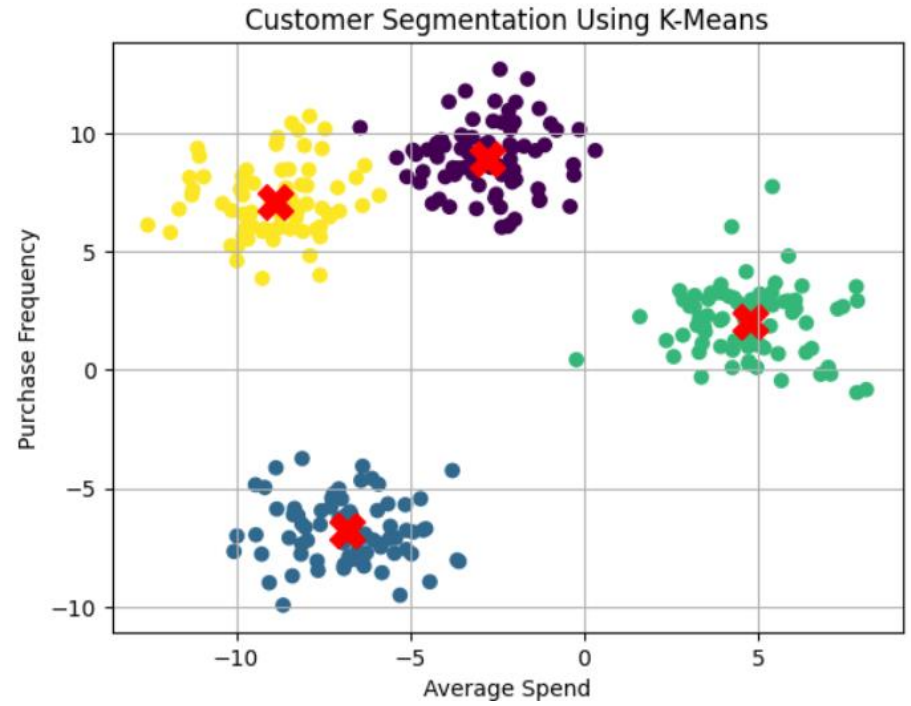
```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        20
           1       1.00      1.00      1.00        20
           2       1.00      1.00      1.00        20

    accuracy                           1.00        60
   macro avg       1.00      1.00      1.00        60
weighted avg       1.00      1.00      1.00        60
```

Case Study 1: Customer Segmentation in Marketing

A retail company wants to improve its targeted advertising. Instead of treating all customers the same, the marketing team wants to group customers based on behavior—such as average spending, frequency of purchase, and product categories.

To achieve this, the data team uses K-Means clustering to segment customers into groups with similar characteristics.

```python
# Case Study 1: Customer Segmentation in Marketing
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import pandas as pd

X, _ = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=1.5,
    random_state=42
)

kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

kmeans.fit(X)

labels = kmeans.predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=250,
    c='red',
    marker='X'
)

plt.title("Customer Segmentation Using K-Means")
plt.xlabel("Average Spend")
plt.ylabel("Purchase Frequency")
plt.grid(True)

plt.show()
```

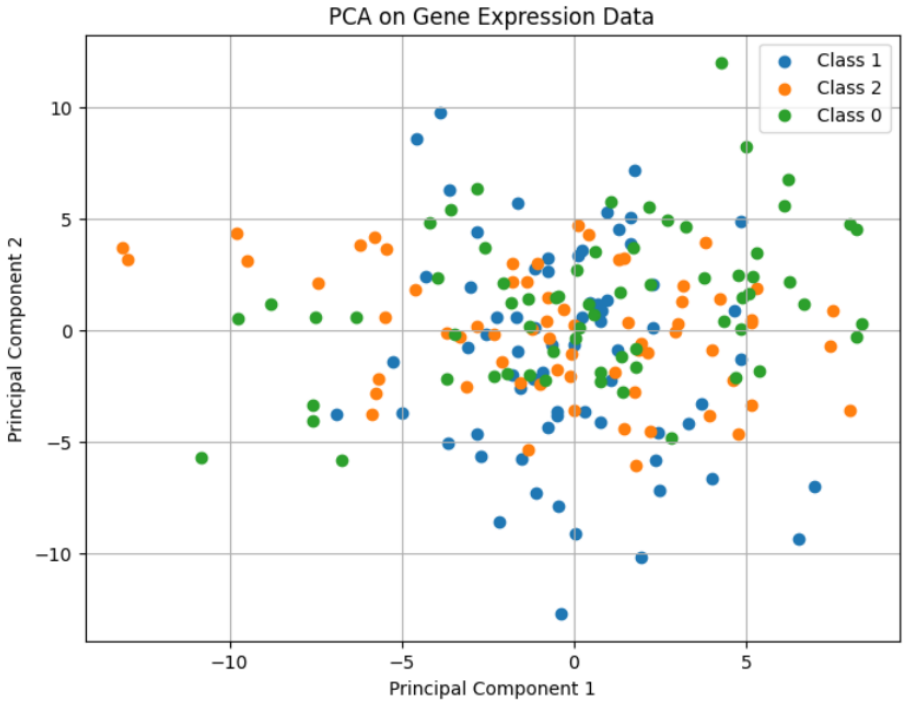


Case Study 2: Genomic Data Compression Using PCA

A bioinformatics lab is working with high-dimensional gene expression data. Each sample contains thousands of gene expression values. For visualization and classification, they need to reduce the dimensionality without losing essential information.

The team uses Principal Component Analysis (PCA) to reduce the dataset from thousands of features to just 2 or 3 principal components.

```
# Case Study 2: Genomic Data Compression Using PCA
from sklearn.decomposition import PCA
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
import pandas as pd

X, y = make_classification(
    n_samples=200,
    n_features=50,
    n_informative=10,
    n_classes=3,
    random_state=42
)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)

df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

df['Class'] = y

plt.figure(figsize=(8, 6))

for label in df['Class'].unique():
    plt.scatter(
        df[df['Class'] == label]['PC1'],
        df[df['Class'] == label]['PC2'],
        label=f'Class {label}'
    )

plt.legend()

plt.title("PCA on Gene Expression Data")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.grid(True)

plt.show()
```



Case Study 3: Handwritten Digit Recognition

A logistics company wants to automatically process zip codes written on packages. This requires recognizing handwritten digits in real time. They decide to use an Artificial Neural Network (ANN) to classify images of handwritten digits.

They use the MNIST dataset to train a basic neural network using MLPClassifier.

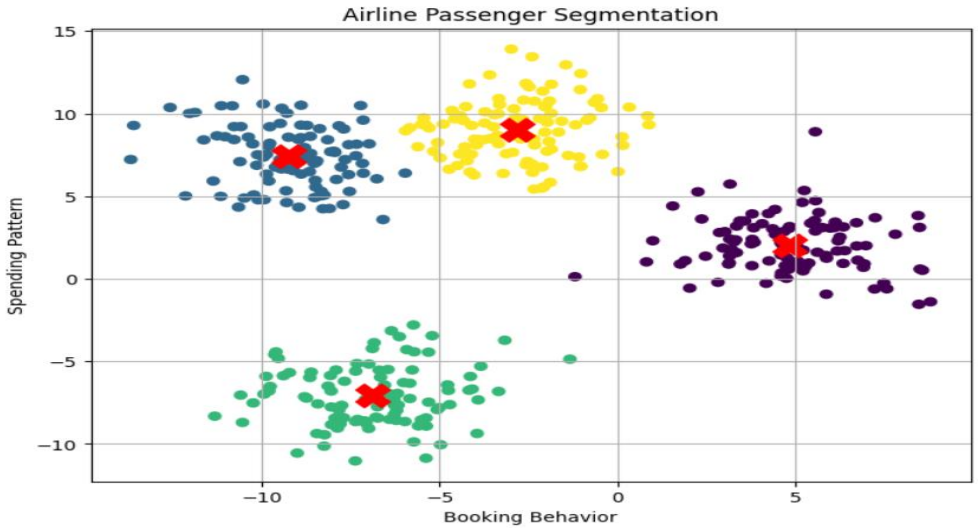
```
# Case Study 3: Handwritten Digit Recognition
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt

digits = load_digits()

X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    max_iter=300,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	53
1	0.96	0.98	0.97	50
2	0.98	1.00	0.99	47
3	0.98	0.96	0.97	54
4	1.00	1.00	1.00	60
5	0.97	0.95	0.96	66
6	0.96	0.96	0.96	53
7	1.00	0.98	0.99	55
8	0.93	0.91	0.92	43
9	0.95	0.97	0.96	59
accuracy			0.97	540
macro avg	0.97	0.97	0.97	540
weighted avg	0.97	0.97	0.97	540

Case Study 1: Dynamic Pricing Optimization in Airline Industry
Technique: Cluster Analysis
□ Scenario: An international airline wants to personalize ticket pricing using customer behavior data, moving beyond traditional booking class segmentation.
□ Objective: Identify hidden passenger segments to optimize dynamic pricing and increase revenue efficiency.

```
# Case Study 1: Dynamic Pricing Optimization in Airline Industry
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import pandas as pd

X, _ = make_blobs(
    n_samples=400,
    centers=4,
    cluster_std=1.8,
    random_state=42
)

kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

kmeans.fit(X)

labels = kmeans.predict(X)

plt.figure(figsize=(8, 6))

plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels,
    cmap='viridis'
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=300,
    c='red',
    marker='X'
)
```

```
plt.title("Airline Passenger Segmentation")
plt.xlabel("Booking Behavior")
plt.ylabel("Spending Pattern")

plt.grid(True)

plt.show()
```



Case Study 2: Early Diagnosis of Parkinson's Disease
Technique: PCA + Artificial Neural Networks
□ Scenario: A healthcare research institute uses voice recordings to detect early signs of Parkinson's, dealing with hundreds of acoustic features per patient.
□ Objective: Reduce data dimensionality and train an accurate neural network classifier for early, non-invasive diagnosis.

```
# Case Study 2: Early Diagnosis of Parkinson's Disease
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

X, y = make_classification(
    n_samples=500,
    n_features=100,
    n_informative=20,
    n_classes=2,
    random_state=42
)

pca = PCA(n_components=20)

X_pca = pca.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_pca,
    y,
    test_size=0.3,
    random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    max_iter=500,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.91      0.91      0.91        76
           1       0.91      0.91      0.91        74

    accuracy                           0.91       150
   macro avg       0.91      0.91      0.91       150
weighted avg       0.91      0.91      0.91       150
```

Case Study 3: Credit Card Fraud Detection at Scale
Technique: ANN + PCA
□ Scenario: A financial company processes millions of credit card transactions and wants to detect fraud patterns hidden in complex, high-dimensional data.
□ Objective: Reduce noise and redundant features using PCA, and apply ANN to classify transactions as fraudulent or genuine in real time.

```python
# Case Study 3: Credit Card Fraud Detection at Scale
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

X, y = make_classification(
    n_samples=1000,
    n_features=50,
    n_informative=15,
    weights=[0.95, 0.05],
    random_state=42
)

pca = PCA(n_components=15)

X_pca = pca.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_pca,
    y,
    test_size=0.3,
    random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(50, 25),
    max_iter=500,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.96      0.99      0.98       282
           1       0.75      0.33      0.46        18

    accuracy                           0.95       300
   macro avg       0.85      0.66      0.72       300
weighted avg       0.95      0.95      0.94       300
```

Case Study 4: Consumer Trend Mapping in Retail Chain
Technique: Cluster Analysis + PCA
□ Scenario: A national retail chain wants to understand evolving customer preferences across regions using transaction data, product choices, and seasonal trends.
□ Objective: Uncover underlying patterns and segment customers based on regional behaviors and product affinity using clustering on PCA-reduced data.

```
# Case Study 4: Consumer Trend Mapping in Retail Chain
from sklearn.datasets import make_blobs
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import pandas as pd

X, _ = make_blobs(
    n_samples=500,
    n_features=10,
    centers=5,
    random_state=42
)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)

kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(X_pca)

df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

df['Cluster'] = labels

plt.figure(figsize=(8, 6))

plt.scatter(
    df['PC1'],
    df['PC2'],
    c=df['Cluster'],
    cmap='Set2'
)

plt.title("Retail Consumer Trend Clusters")
```

```
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.grid(True)

plt.show()
```

Case Study 5: Quality Control in Smart Manufacturing
Technique: ANN
□ Scenario: An electronics manufacturer uses real-time sensor data from assembly lines to detect potential defects. Data includes vibration, temperature, voltage, and acoustic signals.
□ Objective: Train an ANN model to classify products into "pass" or "defect" categories with high accuracy to enable automated quality assurance.

```python
# Case Study 5: Quality Control in Smart Manufacturing
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

X, y = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=10,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    max_iter=500,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.94      0.92      0.93       153
           1       0.92      0.94      0.93       147

    accuracy                           0.93       300
   macro avg       0.93      0.93      0.93       300
weighted avg       0.93      0.93      0.93       300
```

https://github.com/Habiba-Bibi/